FEAT: Fair and Efficient Adversarial Transaction Ordering

Tien Tuan Anh Dinh
anh.dinh@deakin.edu.au
Deakin University
Melbourne, Victoria, Australia

Dakai Kang
dakang@ucdavis.edu
University of California, Davis
Davis, California, USA

Mohammad Sadoghi
msadoghi@ucdavis.edu
University of California, Davis
Davis, California, USA

Abstract

Fair transaction ordering protocols that guarantee γ-Batch-Order-Fairness are a leading defense against Maximal Extractable Value (MEV) attacks in Byzantine consensus and blockchain systems. All existing such protocols build a tournament dependency graph among transactions, incurring $\Theta(|\mathcal{T}|^2)$ comparator queries per round on a transaction set $\mathcal{T}$, which becomes the dominant performance bottleneck.

We resolve the comparison complexity of enforcing γ-Batch-Order-Fairness. Representing fairness constraints as a global pairwise comparator that may exhibit non-transitive cycles, we introduce *Adjacent-Comparator Consistency* as a sufficient condition for γ-Batch-Order-Fairness under the γ-induced comparator, and prove a lower bound of $\Omega(|\mathcal{T}|\log|\mathcal{T}|)$ comparator queries in the worst case over comparators via reduction from comparison-based sorting. We then present FEAT, an AVL-tree-based fair-ordering algorithm that achieves the matching $O(|\mathcal{T}|\log|\mathcal{T}|)$ upper bound: rather than building a tournament dependency graph, FEAT inserts transactions into an AVL Tree, consulting the comparator only along an $O(\log|\mathcal{T}|)$-length root-to-leaf path per transaction. Together, these bounds pin the complexity of enforcing γ-Batch-Order-Fairness at $\Theta(|\mathcal{T}|\log|\mathcal{T}|)$.

CCS Concepts

• **Security and privacy** → **Distributed systems security**; • **Computer systems organization** → *Reliability*.

Keywords

Fair transaction ordering, Byzantine fault tolerance, Maximal Extractable Value, Blockchain

ACM Reference Format:
Tien Tuan Anh Dinh, Dakai Kang, and Mohammad Sadoghi. 2026. FEAT: Fair and Efficient Adversarial Transaction Ordering. In *ACM Symposium on Principles of Distributed Computing (PODC '26), July 06–10, 2026, Egham, United Kingdom.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3796701.3815905

1 Introduction

Transaction ordering lies at the heart of Byzantine consensus [5, 15, 38, 45] and blockchain systems [1, 8, 9, 11, 12, 33, 36, 43]. Beyond safety and liveness, modern blockchain applications, particularly Decentralized Finance (DeFi) [39, 47], increasingly demand fairness guarantees, ensuring that transactions are ordered consistently with their observed arrival order. Absent such guarantees, adversarial participants may manipulate ordering to extract profit, a phenomenon commonly referred to as Maximal Extractable Value (MEV) attacks [18, 44]. Uncontrolled MEV undermines both economic fairness and system trustworthiness.

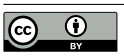
Several prior works aim to achieve fair transaction ordering by constructing a final ordering from the local orderings collected across a majority of replicas. A natural fairness notion in this line of work is *Receive-Order-Fairness*: if sufficiently many honest replicas receive a transaction T before another transaction T', then all honest replicas must output T before T'. However, Aequitas [29] shows that Receive-Order-Fairness is impossible in general. The impossibility arises from the existence of *Condorcet cycles* [10, 16], where non-transitive transaction precedence relations form cycles (see Figure 1 for an example). To circumvent this limitation, Aequitas introduces a weaker notion, *γ-Batch-Order-Fairness*, which partitions the final ordering into batches, allowing transactions within a cycle to be finalized together and requiring T to appear in a batch *no later* than T'.

Definition 1.1. γ-Batch-Order-Fairness: For any two transactions T_i and T_j in $\mathcal{T}$, if at least a fraction γ of honest replicas receive T_i before T_j, then T_i must be ordered *no later* than T_j in the final transaction ordering of $\mathcal{T}$.

More recent protocols guarantee the fairness notion and improve based on Aequitas. In particular, Themis [28] introduces round-based *batch spooling* to address a weak-liveness issue in Aequitas, and it requires $n > 4f$ replicas to tolerate f adversarial replicas. Quick-Order-Fairness [14] and FairDAG-RL [24] reduce this requirement to $n > 3f$, with FairDAG-RL further leveraging DAG-based consensus to improve performance.

We observe that all existing fair-ordering protocols that guarantee *γ-Batch-Order-Fairness* perform a common step, that is, they all construct a tournament dependency graph, which is a complete pairwise precedence relation among transactions. Maintaining and processing such complete pairwise relations incurs quadratic complexity in the size $|\mathcal{T}|$ of the transaction set. As observed in FairDAG [24], with a high-throughput consensus layer [7, 19, 21, 22, 26, 40] such as multi-proposer designs with batching mechanisms, this quadratic dependency-graph construction becomes a dominant performance bottleneck of fair-ordering protocols across all network settings, from close-network deployments to geo-distributed ones, since the per-round $|\mathcal{T}|$ grows large enough that the $\Theta(|\mathcal{T}|^2)$ work outweighs the network cost regardless of latency. This raises the following fundamental question: *Is the quadratic cost inherent to enforcing γ-Batch-Order-Fairness, or merely an artifact of current protocol designs?*

All prior protocols enforcing γ-Batch-Order-Fairness explicitly build the tournament dependency graph among transactions, which makes the $\Theta(|\mathcal{T}|^2)$ cost appear inherent. Our key insight is that



This work is licensed under a Creative Commons Attribution 4.0 International License.
PODC '26, Egham, United Kingdom
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2512-8/2026/07
https://doi.org/10.1145/3796701.3815905

$R_1 : \{T_1, T_2, T_3\}$
$R_2 : \{T_2, T_3, T_1\}$
$R_3 : \{T_3, T_1, T_2\}$

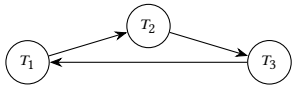


Figure 1: Condorcet Cycle Example. Consider 3 replicas R_1, R_2, R_3 and 3 transactions T_1, T_2, T_3, it is impossible to have a final ordering that, for each transaction T_i and T_j that a majority (2/3) of replicas receive T_i before T_j, orders T_i before T_j.

the final ordering can be assembled without ever building the full tournament graph: by inserting transactions into a balanced binary search tree, the dependency comparator is consulted only along an $O(\log |\mathcal{T}|)$-length root-to-leaf path. Realized via an AVL tree, this yields a fair-ordering algorithm with $O(|\mathcal{T}| \log |\mathcal{T}|)$ comparison complexity. A matching $\Omega(|\mathcal{T}| \log |\mathcal{T}|)$ lower bound, obtained by reduction from comparison-based sorting, pins the complexity of enforcing γ-Batch-Order-Fairness at $\Theta(|\mathcal{T}| \log |\mathcal{T}|)$.

Specifically, we make the following contributions:

(1) To prove the complexity lower bound, we define *γ-Induced Comparator* and *Adjacent-Comparator Consistency* such that *Adjacent-Comparator Consistency* is a sufficient condition for γ-Batch-Order-Fairness with a *γ-Induced Comparator*.
(2) We show that enforcing *Adjacent-Comparator Consistency* incurs a comparison complexity of $\Omega(|\mathcal{T}| \log |\mathcal{T}|)$ in the worst case over comparators. Thus, we prove that guaranteeing γ-Batch-Order-Fairness by enforcing *Adjacent-Comparator Consistency* has comparison complexity lower bound $\Omega(|\mathcal{T}| \log |\mathcal{T}|)$ in the worst-case over comparators.
(3) We present FEAT, a fair-ordering algorithm that implements a *γ-Induced Comparator*, guarantees *Adjacent-Comparator Consistency*, and achieves $O(|\mathcal{T}| \log |\mathcal{T}|)$ comparison complexity via an AVL-tree-based construction [2], thereby proving tightness.

The remainder of the paper is organized as follows. Section 2 discusses related work. Section 3 presents the system model. Section 4 formalizes the fairness problem. Section 5 describes the design of FEAT. Section 6 establishes correctness and tight comparison complexity bounds of FEAT. Finally, Section 7 concludes.

2 Related Work

Existing fair-ordering protocols that guarantee the fairness property γ-Batch-Order-Fairness, including Aequitas [29], Themis [28], Quick-Order-Fairness [14], and FairDAG-RL [24], calculate a complete ordering relation among transactions, leading to quadratic comparison complexity in the transaction set size $|\mathcal{T}|$. At a high level, these protocols construct a tournament dependency graph over transactions, in which exactly one directed edge is determined for each transaction pair. Constructing such a tournament dependency graph incurs quadratic comparison complexity in the transaction set size $|\mathcal{T}|$. They then identify the strongly connected components of the graph, where each component consists of transactions that are mutually reachable via directed paths. The components are subsequently ordered through a topological sort, and transactions within each component are linearized in a manner consistent with the pairwise precedence constraints, yielding the final transaction ordering, e.g., finding a Hamilton path. Figure 2 illustrates this on a 7-transaction example with two strongly connected components.

Beyond the protocols discussed above, some prior works pursue fair transaction ordering with alternative approaches. Wendy [30], Pompe [48], and FairDAG-AB [24] provide a weaker fairness guarantee, namely *Ordering Linearizability*. DCN [17] guarantees *δ-Median Fairness*, which relaxes strict majority-based precedence but requires strong timing assumptions. Rashnu [35] reduces the comparison complexity of Themis by enforcing *γ-Batch-Order-Fairness* only among data-dependent transactions, but incurs a quadratic complexity to detect data dependencies between transactions.

Some protocols seek to eliminate the block proposers' oligarchy over the ordering of transactions within blocks via *censorship resistance* [3, 6, 13, 20, 31, 32, 34, 37, 41, 42, 46]. In these protocols, a transaction is encrypted until the ordering of the transaction is determined. However, block proposers can still engage in censorship based on metadata, such as IP addresses, or prioritize their own transactions whose content they know.

A complementary line of work mitigates reordering attacks via random permutation: Eating Sandwiches [4] applies a commit-reveal random permutation with transaction chunking, and BlindPerm [27] composes an encrypted mempool with a randomized post-commit permutation.

Other approaches operate at the consensus layer: HotStuff-1 [25] uses a slotting mechanism to narrow the proposer's MEV window, Carry-the-Tail [23] mitigates tail-forking MEV by carrying pending transactions across view changes, and multi-proposer protocols [7, 19, 21, 22, 26] distribute ordering control across concurrent proposers so that no single Byzantine proposer can unilaterally manipulate the order.

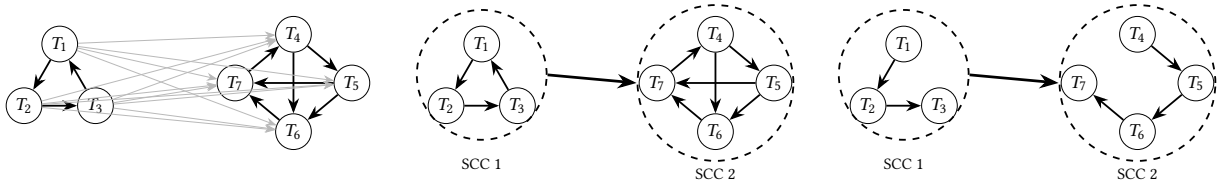
3 System Model

3.1 Replicas and Fault Model.

We consider a distributed system consisting of a fixed set of n replicas. Up to f replicas may be *Byzantine/Adversarial*, exhibiting arbitrary malicious behavior, while the remaining replicas are *honest* and follow the protocol specification.

Transactions are disseminated through authenticated broadcast. Each replica locally observes a sequence of transactions, which may differ across replicas due to network delays or adversarial interference. Honest replicas broadcast their local transaction ordering honestly, one fragment per round, while Byzantine replicas may broadcast arbitrary sequences in an attempt to manipulate the final transaction ordering. We assume an authenticated communication model in which Byzantine replicas cannot forge messages on behalf of honest replicas.

The system tolerates Byzantine faults under the condition $n > \frac{(2\gamma+1)f}{2\gamma-1}$, where γ is a configurable system parameter satisfying $0.5 < \gamma \leq 1$. Intuitively, γ controls the fraction of ordering information required to enforce fairness constraints. When $\gamma = 1$, the requirement reduces to the classical requirement $n > 3f$.



(a) Tournament dependency graph (b) Strongly connected components, topologically sorted (c) Linearized within each component

Figure 2: The three steps in previous fair-ordering protocols that guarantee γ-*Batch-Order-Fairness*, illustrated on a 7-transaction example. (a) The *tournament dependency graph*: each transaction pair has exactly one directed edge encoding the pairwise precedence relation. Transactions $\{T_1, T_2, T_3\}$ and $\{T_4, T_5, T_6, T_7\}$ each form a strongly connected component (SCC) (mutually reachable via directed paths), and every transaction in $\{T_1, T_2, T_3\}$ precedes every transaction in $\{T_4, T_5, T_6, T_7\}$ (cross edges in gray). Building this graph requires one comparator query per transaction pair, $\binom{7}{2} = 21$ in total. (b) The strongly connected components SCC 1 and SCC 2 are condensed (dashed circles) and topologically sorted. (c) Within each component, transactions are linearized with the pairwise precedence constraints (a Hamilton path through the cycle), yielding the final ordering $T_1, T_2, T_3, T_4, T_5, T_6, T_7$.

3.2 Underlying Consensus Layer

Since this work focuses on the transaction ordering algorithm after local orderings are committed, we assume the existence of an underlying Byzantine fault-tolerant consensus layer. This layer commits local ordering fragments proposed by replicas for the ordering layer. It is treated as a black box and required to satisfy the following properties.

(1) **Round-based operation.** The consensus protocol proceeds in logical rounds indexed by $r = 1, 2, \ldots$. Each round produces a consensus decision.
(2) **Consensus on local ordering fragments.** In each round r, replicas propose local transaction ordering fragments honestly. The consensus layer commits ordering fragments from at least $n - f$ replicas every round.
(3) **Sequential inclusion of honest replicas' proposals.** For every honest replica, all of its local ordering fragments are eventually included in the consensus decisions, in the strict order in which the transactions are observed by the honest replica.
(4) **Consistency of transaction visibility among honest replicas.** If a transaction T appears in the local ordering of any honest replica, then it eventually appears in the local ordering of all honest replicas, and all the honest local ordering fragments containing T are committed within at most c rounds, for some constant c.

These assumptions abstract away the details of the underlying consensus mechanism and capture the minimal guarantees required by the ordering layer. Given these guarantees, the ordering layer is responsible for producing a final transaction ordering that satisfies γ-Batch-Order-Fairness.

4 Problem Statement

4.1 Pairwise Precedence Relation

Our goal is to derive a single global transaction ordering that is *fair* with respect to the local orderings of honest replicas. The term "no later" in Definition 1.1 is underspecified. In fact, it implies that a pairwise precedence relation may be non-transitive, and that the maximal sets of transactions mutually reachable under this relation form cycles, which we refer to as "batches".

We model the pairwise precedence relations using a global transaction precedence comparator

$$\mathrm{cmp}_{\mathcal{T}} : \mathcal{T} \times \mathcal{T} \to \{\mathsf{True}, \mathsf{False}\},$$

For distinct transactions $T \neq T'$, the value $\mathrm{cmp}_{\mathcal{T}}(T, T') = \mathsf{True}$ indicates that fairness requires T to be ordered before T'; similarly, $\mathrm{cmp}_{\mathcal{T}}(T, T') = \mathsf{False}$ indicates that fairness requires T' to be ordered before T. The comparator induces a *tournament*: for distinct $T, T' \in \mathcal{T}$, $\mathrm{cmp}_{\mathcal{T}}(T, T')$ and $\mathrm{cmp}_{\mathcal{T}}(T', T)$ are negations of each other, so the precedence between any pair is unambiguously determined. Transitivity, however, is not required, and therefore precedence cycles may arise.

Round-by-round revelation. During consensus protocol execution, the ordering layer does not necessarily know the precedence relation for every transaction pair, because local ordering fragments are committed round by round by the underlying consensus protocol. For a transaction pair, the precedence relation may temporarily remain undetermined until more local ordering fragments are committed.

Instead, committed local orderings up to round r induce a round-specific partial comparator

$$\widehat{\mathrm{cmp}}_{\mathcal{T}}^{(r)} : \mathcal{T} \times \mathcal{T} \rightharpoonup \{\mathsf{True}, \mathsf{False}\},$$

which captures the precedence relations that have become supported by the local ordering fragments committed up to round r.

We assume monotone refinement: once a precedence relation between a transaction pair becomes supported by committed evidence, it is fixed and will not be revised in later rounds. Formally, if for some round r, $\widehat{\mathrm{cmp}}_{\mathcal{T}}^{(r)}(T, T')$ is defined, then for all $r' > r$, $\widehat{\mathrm{cmp}}_{\mathcal{T}}^{(r')}(T, T') = \widehat{\mathrm{cmp}}_{\mathcal{T}}^{(r)}(T, T')$.

Accordingly, we assume that the global comparator is eventually complete. And the value for each pair is defined by the first round in which the pair becomes determined. Formally, for any transaction pair (T, T'), let $r^\star$ be the smallest round such that $\widehat{\mathrm{cmp}}_{\mathcal{T}}^{(r^\star)}(T, T')$ is defined. Then we define $\mathrm{cmp}_{\mathcal{T}}(T, T') = \widehat{\mathrm{cmp}}_{\mathcal{T}}^{(r^\star)}(T, T')$.

4.2 Comparator-Based Formulation of γ-Batch-Order-Fairness

Comparator-Based Batches. Given the global comparator $\text{cmp}_{\mathcal{T}} : \mathcal{T} \times \mathcal{T} \to \{\text{True}, \text{False}\}$, we define a directed relation over transactions as follows:

$$T \to T' \quad \text{iff} \quad \text{cmp}_{\mathcal{T}}(T, T') = \text{True}.$$

Let $\rightsquigarrow$ denote reachability under this relation, i.e., $T \rightsquigarrow T'$ if there exists a directed path from T to T'. Then we define an equivalence relation $\sim$ over $\mathcal{T}$ by

$$T \sim T' \quad \text{iff} \quad (T \rightsquigarrow T') \wedge (T' \rightsquigarrow T).$$

A *batch* is an equivalence class of $\mathcal{T}$ under $\sim$; equivalently, within a batch there exists a directed path from any transaction to any other transaction.

Definition 4.1 (Comparator-Based γ-Batch-Ordering-Fairness). Let FO be the final ordering over $\mathcal{T}$. We say that FO satisfies comparator-based γ-Batch-Ordering-Fairness if the following conditions hold:

(1) **Linear Batch Partition.** There exist $k \in \mathbb{N}$ and a partition $\mathcal{T} = B_1 \sqcup B_2 \sqcup \cdots \sqcup B_k$ into non-empty contiguous batches such that FO enumerates the elements of B_1, then B_2, ..., then B_k (within each B_i in some order), and

$$\forall\, 1 \le i < j \le k,\ \forall T \in B_i,\ \forall T' \in B_j : \text{cmp}_{\mathcal{T}}(T, T') = \text{True}.$$

(2) **γ-Precedence Respect.** Let $batch : \mathcal{T} \to \{1, 2, \ldots, k\}$ map each transaction to the index of the batch containing it. For any $T, T' \in \mathcal{T}$, if at least a γ-fraction of honest replicas receive T before T', then $batch(T) \le batch(T')$.

4.3 A Sufficient Condition for γ-Batch-Order-Fairness

Although the comparator-based γ-Batch-Order-Fairness provides a characterization of fairness, its definition permits arbitrary ordering within each batch. To enable a cleaner lower-bound argument, we introduce a stronger pairwise precedence property that is sufficient to guarantee γ-Batch-Order-Fairness with respect to a *γ-induced comparator*.

Definition 4.2 (γ-induced comparator). The comparator $\text{cmp}_{\mathcal{T}}$ is *γ-induced* if, for any $T, T' \in \mathcal{T}$, at least a γ-fraction of honest replicas receiving T before T' implies $\text{cmp}_{\mathcal{T}}(T, T') = \text{True}$.

Definition 4.3 (Adjacent-Comparator Consistency). Let FO be the final ordering over $\mathcal{T}$. We say that FO satisfies *Adjacent-Comparator Consistency* (with respect to $\text{cmp}_{\mathcal{T}}$) if for every position i with $1 \le i < |\mathcal{T}|$, the comparator on adjacent transactions equals True:

$$\text{cmp}_{\mathcal{T}}\big(\text{FO}[i], \text{FO}[i+1]\big) = \text{True}.$$

Theorem 4.4 (Sufficiency of Adjacent-Comparator Consistency). *Assume the global comparator $\text{cmp}_{\mathcal{T}} : \mathcal{T} \times \mathcal{T} \to \{True, False\}$ is γ-induced. Let* FO *be the final ordering over $\mathcal{T}$. If* FO *satisfies Adjacent-Comparator Consistency, then* FO *satisfies comparator-based γ-Batch-Order-Fairness.*

Proof. *Preliminary.* By Adjacent-Comparator Consistency, it holds that $\text{FO}[i] \to \text{FO}[i+1]$ for every i. Hence for any indices $a < b$, there exists a directed path $\text{FO}[a] \to \text{FO}[a+1] \to \cdots \to \text{FO}[b]$, so $\text{FO}[a] \rightsquigarrow \text{FO}[b]$.

(1) Linear batch partition (by contradiction). We first show that contiguous batches form a linear partition of FO and that cross-batch comparator directions are forward.

Batch contiguity. Suppose for contradiction that some batch is not contiguous in FO. Then there exist indices $a < b < c$ such that $\text{FO}[a] \sim \text{FO}[c]$ but $\text{FO}[b] \not\sim \text{FO}[a]$. From the preliminary fact, $\text{FO}[a] \rightsquigarrow \text{FO}[b]$ and $\text{FO}[b] \rightsquigarrow \text{FO}[c]$. Also $\text{FO}[c] \rightsquigarrow \text{FO}[a]$ since $\text{FO}[a] \sim \text{FO}[c]$. Thus $\text{FO}[b] \rightsquigarrow \text{FO}[a]$, and together with $\text{FO}[a] \rightsquigarrow \text{FO}[b]$ we obtain $\text{FO}[a] \sim \text{FO}[b]$, contradicting $\text{FO}[b] \not\sim \text{FO}[a]$. Therefore FO can be linearly partitioned into contiguous batches.

Cross-batch direction. Consider any two batches B_i and B_j with $i < j$. Take any positions a in B_i and b in B_j (so $a < b$ in FO). Suppose for contradiction that $\text{cmp}_{\mathcal{T}}(\text{FO}[a], \text{FO}[b]) = \text{False}$. By anti-symmetry, this implies $\text{cmp}_{\mathcal{T}}(\text{FO}[b], \text{FO}[a]) = \text{True}$, i.e., $\text{FO}[b] \to \text{FO}[a]$, hence $\text{FO}[b] \rightsquigarrow \text{FO}[a]$. But from the preliminary fact, $\text{FO}[a] \rightsquigarrow \text{FO}[b]$. Thus $\text{FO}[a] \sim \text{FO}[b]$, meaning they belong to the same batch, contradicting that a is in B_i and b is in B_j with $i < j$. Therefore, for any $i < j$, for all $T \in B_i$ and $T' \in B_j$, it must hold that $\text{cmp}_{\mathcal{T}}(T, T') = \text{True}$.

(2) γ-precedence respect (by contradiction). Let $T, T' \in \mathcal{T}$ and suppose at least a γ-fraction of honest replicas receive T before T'. Since $\text{cmp}_{\mathcal{T}}$ is γ-induced, we have $\text{cmp}_{\mathcal{T}}(T, T') = \text{True}$, i.e., $T \to T'$, and hence $T \rightsquigarrow T'$. Suppose for contradiction that $batch(T) > batch(T')$, i.e., $T \in B_j$ and $T' \in B_i$ for some $i < j$. By the linear batch partition established above, T' occupies an earlier position than T in FO; then by the preliminary fact, $T' \rightsquigarrow T$. Combined with $T \rightsquigarrow T'$, this gives $T \sim T'$, forcing T and T' to lie in the same batch—a contradiction. Therefore $batch(T) \le batch(T')$.

Summary. Parts (1) and (2) establish, respectively, the two clauses of the comparator-based γ-Batch-Order-Fairness definition: FO admits a contiguous batch partition $\mathcal{T} = B_1 \sqcup B_2 \sqcup \cdots \sqcup B_k$ with forward cross-batch comparator directions (Linear Batch Partition), and the induced index function $batch$ satisfies $batch(T) \le batch(T')$ whenever at least a γ-fraction of honest replicas receive T before T' (γ-Precedence Respect). Hence FO satisfies comparator-based γ-Batch-Order-Fairness, as required.

□

4.4 Lower Bound for Adjacent-Comparator Consistency

Enforcing Adjacent-Comparator Consistency requires constructing a final ordering whose adjacent transaction pairs respect the global comparator. When the global comparator is transitive, this requirement is equivalent to producing the unique sorted order of the transactions. Since transitive comparators form a subset of all tournament comparators, the worst comparator for enforcing Adjacent-Comparator Consistency is at least as hard as sorting under a transitive comparator, which requires $\Omega(|\mathcal{T}| \log |\mathcal{T}|)$ comparisons, as proved below.

Theorem 4.5 (Expected lower bound in the worst-case over comparators). *Let A be any comparison-based algorithm that, given a transaction set $\mathcal{T}$ together with oracle access to a tournament comparator $\text{cmp}_{\mathcal{T}}$ (i.e., for all $T \neq T'$ in $\mathcal{T}$, exactly one of $\text{cmp}_{\mathcal{T}}(T, T')$ and $\text{cmp}_{\mathcal{T}}(T', T)$ equals True), outputs a final ordering* FO *of $\mathcal{T}$*

satisfying Adjacent-Comparator Consistency*:*

$$\mathrm{cmp}_{\mathcal{T}}(\mathrm{FO}[i], \mathrm{FO}[i+1]) = \mathit{True} \quad \mathit{for\ all}\ i \in \{1, \ldots, |\mathcal{T}| - 1\}.$$

Assume the input arrival order is a uniformly random permutation $\pi \sim \mathrm{Unif}(S_{\mathcal{T}})$*, where* $S_{\mathcal{T}}$ *denotes the set of all* $|\mathcal{T}|!$ *permutations of* $\mathcal{T}$*, and let* $T_A(\mathrm{cmp}_{\mathcal{T}}, \pi)$ *be the number of comparator queries that* A *makes on input* $(\mathrm{cmp}_{\mathcal{T}}, \pi)$*. Then*

$$\max_{\mathrm{cmp}_{\mathcal{T}}} \mathbb{E}_{\pi}\left[T_A(\mathrm{cmp}_{\mathcal{T}}, \pi)\right] = \Omega(|\mathcal{T}| \log |\mathcal{T}|).$$

Proof. It suffices to exhibit one tournament comparator under which A's expected query count is $\Omega(|\mathcal{T}| \log |\mathcal{T}|)$, since the maximum over all comparators is at least this value.

Fix the *transitive* tournament comparator $\mathrm{cmp}^{\star}_{\mathcal{T}}$ induced by a total order $T_1 < T_2 < \cdots < T_{|\mathcal{T}|}$ on $\mathcal{T} = \{T_1, \ldots, T_{|\mathcal{T}|}\}$:

$$\mathrm{cmp}^{\star}_{\mathcal{T}}(T_i, T_j) = \mathsf{True} \iff i < j.$$

Under $\mathrm{cmp}^{\star}_{\mathcal{T}}$, Adjacent-Comparator Consistency forces $\mathrm{FO}[i] < \mathrm{FO}[i+1]$ for every i, and by transitivity the output ordering is uniquely determined:

$$\mathrm{FO} = (T_1, T_2, \ldots, T_{|\mathcal{T}|}).$$

Hence, on input $(\mathrm{cmp}^{\star}_{\mathcal{T}}, \pi)$, algorithm A must produce the sorted order of $\mathcal{T}$, and each comparator query $\mathrm{cmp}^{\star}_{\mathcal{T}}(T_i, T_j)$ is equivalent to the standard "$T_i < T_j$?" comparison. Thus A instantiates a comparison-based sorting algorithm on the input permutation π.

By the classical information-theoretic lower bound for comparison-based sorting, any such algorithm performs at least $\log_2(|\mathcal{T}|!) = \Omega(|\mathcal{T}| \log |\mathcal{T}|)$ comparisons in expectation under a uniformly random input permutation. Therefore

$$\mathbb{E}_{\pi}\left[T_A(\mathrm{cmp}^{\star}_{\mathcal{T}}, \pi)\right] = \Omega(|\mathcal{T}| \log |\mathcal{T}|),$$

and taking the maximum over all tournament comparators yields

$$\max_{\mathrm{cmp}_{\mathcal{T}}} \mathbb{E}_{\pi}\left[T_A(\mathrm{cmp}_{\mathcal{T}}, \pi)\right] \geq \mathbb{E}_{\pi}\left[T_A(\mathrm{cmp}^{\star}_{\mathcal{T}}, \pi)\right] = \Omega(|\mathcal{T}| \log |\mathcal{T}|).$$

□

Corollary 4.6. *Any comparison-based algorithm that enforces Adjacent-Comparator Consistency makes* $\Omega(|\mathcal{T}| \log |\mathcal{T}|)$ *expected comparator queries in the worst case over comparators. Consequently, any comparison-based enforcement of* γ*-Batch-Order-Fairness that relies on Adjacent-Comparator Consistency has the same lower bound.*

5 FEAT: AVL-Tree-Based Fair-Ordering Algorithm

In this section, we present FEAT, an AVL-tree-based fair-ordering algorithm. Unlike the previous tournament-construction algorithm [14, 24, 28, 29] that explicitly performs comparison for every transaction pair, FEAT performs comparisons only as needed to determine the final ordering.

FEAT sits in the *fair-ordering layer* of the two-layer architecture for fair transaction ordering protocols shown in Figure 3. In each round, the consensus layer commits local-ordering fragments; the fair-ordering layer, which FEAT occupies, uses these commits to produce a fair final ordering. FEAT is therefore a drop-in replacement for the fair-ordering component of existing protocols, leaving the choice of underlying consensus protocol orthogonal to the fairness guarantee. Some prior protocols, e.g., Themis [28], place the *fair-ordering layer* before the *consensus layer* rather than after; the fair-ordering algorithm in that layer can still be replaced by FEAT, since FEAT is independent of the order of the two layers.

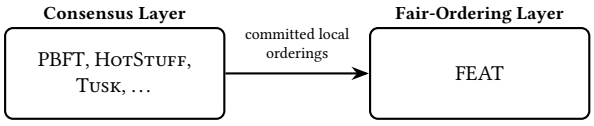


Figure 3: Two-layer architecture for fair-ordering protocols. Each round, the consensus layer (e.g., PBFT**) commits local-ordering fragments; the fair-ordering layer,** FEAT **and algorithms in prior works (e.g.,** Themis **and** Pompe**) uses these commits to generate a fair final ordering.**

In Section 6, we prove that FEAT satisfies *Adjacent-Comparator Consistency* and incurs $O(|\mathcal{T}| \log |\mathcal{T}|)$ per-round comparison complexity, where $\mathcal{T}$ denotes the set of transactions inserted into the round's AVL tree. Combined with the lower bound established earlier, this yields a tight $\Theta(|\mathcal{T}| \log |\mathcal{T}|)$ per-round comparison complexity.

5.1 Building the AVL Tree of Transactions

Pairwise weights. For each round r, let $W_r(T_i, T_j)$ denote the number of committed local orderings (up to and including round r) that place transaction T_i before T_j. As additional local ordering fragments are committed, these weights are monotonically non-decreasing.

Weight-based comparator. Given a decidability threshold $\tau_{\mathrm{dec}} > 0$, we define a round-indexed comparator $\mathrm{pcmp}^{(r)}_{\mathcal{T}} : \mathcal{T} \times \mathcal{T} \to \{\mathsf{True}, \mathsf{False}, \perp\}$ as follows for distinct transactions $T_i \neq T_j$:

(1) If $W_r(T_i, T_j) < \tau_{\mathrm{dec}}$ and $W_r(T_j, T_i) < \tau_{\mathrm{dec}}$, then it is *undecidable*, i.e., $\mathrm{pcmp}^{(r)}_{\mathcal{T}}(T_i, T_j) = \perp$.
(2) If $W_r(T_i, T_j) > W_r(T_j, T_i)$ and $W_r(T_i, T_j) \geq \tau_{\mathrm{dec}}$, then $\mathrm{pcmp}^{(r)}_{\mathcal{T}}(T_i, T_j) = \mathsf{True}$.
(3) If $W_r(T_i, T_j) < W_r(T_j, T_i)$ and $W_r(T_j, T_i) \geq \tau_{\mathrm{dec}}$, then $\mathrm{pcmp}^{(r)}_{\mathcal{T}}(T_i, T_j) = \mathsf{False}$.
(4) If both weights exceed the threshold and are equal, a deterministic hash function is used as a tie-breaker.

Since the underlying consensus layer guarantees that each transaction eventually appears in the committed local orderings of at least $n - f$ honest replicas, we set the decidability threshold to $\tau_{\mathrm{dec}} = \frac{n-f}{2}$, which will eventually be reached.

When calculating $\mathrm{pcmp}^{(r)}_{\mathcal{T}}(T_i, T_j)$ in round r, the protocol first checks whether the precedence between (T_i, T_j) has already been decided in some previous round. If so, the previously determined result is returned directly, and the current weights are ignored. Only if the pair has not been decided before does the comparator calculate the round-r weights to determine the outcome. This requirement ensures that once a precedence relation is established, it is never revised in later rounds, which aligns with our description of the global comparator $\mathrm{cmp}_{\mathcal{T}}$ in Section 4.1.

Algorithm 1: BuildAVLTreeInRound(r)

Input: Round index r. Committed local orderings up to each round.
Output: Round ordering RO_r obtained from the in-order traversal.

```
// Step 1: identify newly non-blank transactions in round r
NewSolid ← {T : T is newly solid in round r}
NewShaded ← {T : T is newly shaded in round r}
ρ ← null;  Deferred ← [ ]
// Step 2: insert newly solid transactions
foreach T ∈ NewSolid do
    ρ ← Insert(ρ, T, r)
// Step 3: attempt to insert newly shaded transactions; defer on ⊥
foreach T ∈ NewShaded do
    ok ← TryInsert(ρ, T, r)
    if ok = False then
        Deferred.append(T)
// Step 4: wait-and-retry deferred shaded transactions in subsequent rounds
r' ← r
while |Deferred| > 0 do
    r' ← r' + 1
    WaitForCommit(r')
    Deferred' ← [ ]
    foreach T ∈ Deferred do
        ok ← TryInsert(ρ, T, r')
        if ok = False then
            Deferred'.append(T)
    Deferred ← Deferred'
// Step 5: return in-order traversal as round ordering
RO_r ← InOrderTraversal(ρ)
return RO_r
```

Transaction classification. Each transaction T is assigned a score $\mathrm{score}(T)$, defined as the number of honest replicas whose committed local orderings include T. Recall that a pairwise precedence relation becomes decidable once the supporting weight exceeds the threshold $\tau_{\mathrm{dec}} = \frac{n-f}{2}$. We set the classification thresholds as $\tau_{\mathrm{shaded}} = \tau_{\mathrm{dec}}, \tau_{\mathrm{solid}} = 2\,\tau_{\mathrm{dec}}$. Transactions are classified as:

- *solid*: $\mathrm{score}(T) \geq \tau_{\mathrm{solid}}$;
- *shaded*: $\tau_{\mathrm{shaded}} \leq \mathrm{score}(T) < \tau_{\mathrm{solid}}$;
- *blank*: otherwise.

Intuitively, shaded transactions have accumulated sufficient appearance to potentially participate in ordering, while solid transactions have gathered enough support to guarantee decidability against any other transaction.

Algorithm 2: TryInsert(ρ, T, r) (ignore comparator/rebalancing details)

Input: Current AVL root ρ; transaction T; round index r.
Output: True if insertion succeeds; False if aborted due to $\perp$.

```
u ← ρ
while u ≠ null do
    T_u ← u.txn
    q ← pcmp_T^(r)(T, T_u)
    if q = True then
        u ← u.left
    else if q = False then
        u ← u.right
    else
        return False
ρ ← AttachLeafAndRebalance(ρ, T)
return True
```

AVL-tree construction. We generate the final ordering by first constructing an AVL-tree with an empty root ρ in each round r, where the relative position of two transactions is determined by the comparator $\mathrm{pcmp}_{\mathcal{T}}^{(r)}$. We insert only the transactions that are newly classified as non-blank (i.e., solid or shaded) in the current round r. Transactions that were classified as non-blank in earlier rounds are excluded. The complete per-round procedure is shown as Algorithm 1, and TryInsert, the per-transaction insertion subroutine for *shaded transactions* is given as Algorithm 2.

(1) **Insert solid transactions.** We first insert all solid transactions. Since solid transactions are decidable against all other nodes, each insertion succeeds.
(2) **Comparator-guided search.** To insert a transaction T, we traverse the tree from the root. At a visited node storing transaction T_u, we query $q \leftarrow \mathrm{pcmp}_{\mathcal{T}}^{(r)}(T, T_u)$. If $q = \mathsf{True}$, we proceed to the left subtree; otherwise ($q = \mathsf{False}$), we proceed to the right subtree.
(3) **Attach and rebalance.** Once a null child pointer is reached, T is attached as a new leaf. We then retrace the search path, update node heights, and perform standard AVL rotations to restore balance.
(4) **Round-by-round deferral and retries for shaded transactions.** Shaded transactions are inserted using the same procedure. However, the pairwise precedence between two shaded transactions may be undecidable due to insufficient committed ordering information. If, during search, the comparator returns $\perp$, the insertion of a shaded transaction T is aborted and deferred. At the end of a subsequent round r', newly committed local orderings update the weights $W_{r'}$, potentially resolving previously undecidable comparisons. Deferred shaded transactions are retried in later rounds until insertion succeeds.

Note on insertion order. Beyond the requirement that solid transactions be inserted before shaded ones, FEAT makes no further

Algorithm 3: StitchAndFinalize(r, RO_r, TailList, FO)

Input: Round index r; round ordering RO_r; global tail list TailList; global final ordering FO.
Output: Updated (TailList, FO).

$\mathrm{Merged}_r \leftarrow [\,]$; $i \leftarrow 1$; $j \leftarrow 1$

while $i \le |\mathit{TailList}|$ ***and*** $j \le |\mathrm{RO}_r|$ **do**
  $x \leftarrow \mathrm{TailList}[i]$; $y \leftarrow \mathrm{RO}_r[j]$
  if $\mathrm{cmp}_{\mathcal{T}}(x, y) = \mathit{True}$ **then**
    Merged_r.append(x); $i \leftarrow i + 1$
  else if $\mathrm{cmp}_{\mathcal{T}}(y, x) = \mathit{True}$ **then**
    Merged_r.append(y); $j \leftarrow j + 1$
  else
    wait until more local ordering framents are committed in subsequent rounds **continue**

if $i = |\mathit{TailList}|$ **then**
  while $j \le |\mathrm{RO}_r|$ **do**
    Merged_r.append($\mathrm{RO}_r[j]$); $j \leftarrow j + 1$
else if $j = |\mathrm{RO}_r|$ **then**
  while $i \le |\mathit{TailList}|$ **do**
    Merged_r.append(TailList$[i]$); $i \leftarrow i + 1$

// Finalize a prefix up to the last transaction classified solid by round r
$\ell \leftarrow$ LastSolidIndex(Merged_r, r) // Returns 0 if no such a transaction exists
if $\ell \ge 1$ **then**
  for $t \leftarrow 1$ **to** ℓ **do**
    FO.append($\mathrm{Merged}_r[t]$)

TailList $\leftarrow \mathrm{Merged}_r[\ell + 1\,..]$
return (TailList, FO)

assumption about the order in which the newly non-blank transactions of a round are inserted into the AVL tree: any deterministic ordering of the solids, followed by any deterministic ordering of the shadeds, shared by all honest replicas, suffices. In particular, every honest replica may apply the same permutation within each class—e.g., the lexicographic order on transaction identifiers, or a random permutation derived from a global common coin—so that the resulting AVL trees, in-order traversals RO_r, and final ordering FO remain identical across replicas (cf. Theorem 6.9). The per-round comparison bound established in Section 6 holds for any such permutation.

5.2 Finalizing Transaction Ordering

After inserting all new non-blank transactions in round r, an in-order traversal of the AVL tree produces the round ordering RO_r. In Section 6, we prove that RO_r satisfies *Adjacent-Comparator Consistency* (Lemma 6.1).

However, while this property holds *within* each round, it is not automatically guaranteed *across* different rounds. To ensure global consistency of the final transaction ordering, we maintain a *global tail list*, denoted by TailList, which carries over non-finalized transactions across rounds.

Global tail list and round stitching. Algorithm 3 gives the full stitching and finalization procedure described in this subsection. At the beginning of round r, TailList contains all transactions that have not yet been finalized in previous rounds. After computing the round ordering RO_r, we merge TailList and RO_r using the global comparator $\mathrm{cmp}_{\mathcal{T}}$. Since $\mathrm{cmp}_{\mathcal{T}}$ is defined by the earliest round in which a pair becomes decidable, any comparator outcome used during stitching is irrevocable.

Comparator-guided stitching (two-pointer merge). We perform a two-pointer merge. Let i point to the head of TailList, and j point to the head of RO_r. We iteratively construct a merged list Merged_r as follows:

(1) If i reaches the end of TailList, append the remaining suffix of RO_r to Merged_r and terminate.
(2) If j reaches the end of RO_r, append the remaining suffix of TailList to Merged_r and terminate.
(3) Otherwise, let $x \leftarrow \mathrm{TailList}[i]$ and $y \leftarrow \mathrm{RO}_r[j]$.
   - If $\mathrm{cmp}_{\mathcal{T}}(x, y) = \mathrm{True}$, append x to Merged_r and increment i.
   - Else if $\mathrm{cmp}_{\mathcal{T}}(y, x) = \mathrm{True}$, append y to Merged_r and increment j.
   - Otherwise (i.e., the order between x and y has not yet been decided by round r), we defer the merge and restart it until more ordering information is committed and additional comparator outcomes are available in a subsequent round.

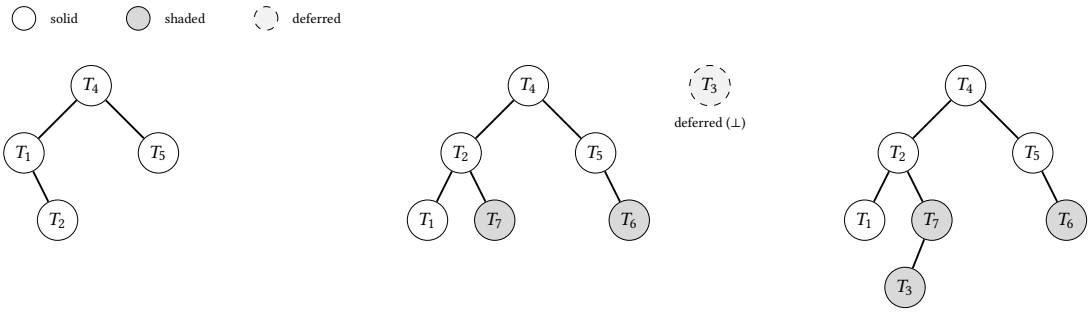
Finalization and tail update. After constructing Merged_r, we output transactions to the global final ordering FO sequentially from its head, until reaching the last transaction that is classified as *solid* by the end of round r. All transactions up to and including this last solid transaction are permanently appended to FO.

The remaining suffix of Merged_r becomes the updated TailList, which will be stitched with RO_{r+1} in the next round.

Figure 4 illustrates a run of FEAT on the 7-transaction example of Figure 2, including the round-by-round deferral and retry mechanism for shaded transactions whose pairwise precedence has not yet become decidable. In this run, the in-order traversal of the final AVL tree is $\mathrm{RO}_{r'} = (T_1, T_2, T_3, T_7, T_4, T_5, T_6)$, in which T_6 is shaded and falls after the last solid transaction T_5. Consequently, the finalization step of round r appends only the prefix up to and including T_5 to the global final ordering FO, while T_6 is carried into TailList to be stitched with RO_{r+1} in the next round.

6 Correctness and Complexity Proof

We prove correctness and complexity tightness of FEAT. First, we show that each round ordering RO_r satisfies Adjacent-Comparator Consistency (Lemma 6.1), that stitching preserves this property (Lemma 6.2), and that consistency holds across round boundaries (Lemma 6.3), implying that the final ordering FO holds Adjacent-Comparator Consistency (Theorem 6.4). Second, we prove that the global comparator in FEAT is γ-induced (Lemma 6.5), which together with the Adjacent-Comparator Consistency yields γ-Batch-Order-Fairness (Theorem 6.6). Finally, we establish that FEAT achieves $O(|\mathcal{T}| \log |\mathcal{T}|)$ per-round comparison complexity (Theorem 6.7)



(a) After inserting solids T_1, T_4, T_5, T_2

(b) Round r: insert shaded T_7, T_6; T_3 aborts at $\mathrm{pcmp}^{(r)}(T_3, T_7) = \perp$

(c) Round $r' > r$: $\mathrm{pcmp}(T_3, T_7) =$ True, T_3 inserted as left child of T_7

Figure 4: Execution of FEAT **on the dependency graph of Figure 2, with insertion order $T_1, T_4, T_5, T_2, T_7, T_3, T_6$ and the latter three transactions classified as *shaded*. We assume that by the end of round r, every pairwise precedence among the transactions is decidable *except* the precedence between T_3 and T_7: that pair has not yet accumulated enough committed local orderings, so $\mathrm{pcmp}_{\mathcal{T}}^{(r)}(T_3, T_7) = \perp$. (a) Solid transactions are inserted first; each insertion succeeds because a solid transaction is decidable against every other transaction. (b) Within round r, shaded T_7 and T_6 are inserted; T_3's insertion is aborted at T_7 because the comparator returns $\perp$, so T_3 is moved to the deferred queue. (c) In a subsequent round $r' > r$, newly committed local orderings resolve $\mathrm{pcmp}(T_3, T_7) =$ True; T_3 is successfully re-inserted as the left child of T_7. The final in-order traversal $T_1, T_2, T_3, T_7, T_4, T_5, T_6$ satisfies Adjacent-Comparator Consistency, and with a total of 13 pairs compared queries: $T_1 : \emptyset$, $T_4 : \{T_1\}$, $T_5 : \{T_1, T_4\}$, $T_2 : \{T_4, T_1\}$, $T_7 : \{T_4, T_1, T_2\}$, $T_6 : \{T_4, T_1\}$, T_3: $\{T_4, T_2, T_7\}$.**

and that this bound is tight (Corollary 6.8). We additionally show that a fair-ordering protocol with FEAT satisfies safety such that every honest replica produces the same final ordering FO (Theorem 6.9).

Lemma 6.1 (Adjacent-Comparator Consistency of round ordering RO_r in FEAT). *For each round r, the in-order traversal ordering RO_r satisfies* Adjacent-Comparator Consistency*: for all i, it holds that $\mathrm{cmp}_{\mathcal{T}}(\mathrm{RO}_r[i], \mathrm{RO}_r[i+1]) =$ True.*

Proof. We prove the claim by induction on k, the number of inserted transaction nodes.

Base case. For $k = 0$ or $k = 1$, the tree has at most one node, and $\mathrm{RO}_r^{(k)}$ has length at most one. Hence there are no adjacent pairs, and the claim holds trivially.

Induction hypothesis. Assume that after inserting k nodes (and performing all necessary AVL rebalancing), the in-order traversal $\mathrm{RO}_r^{(k)}$ satisfies $\forall i, \mathrm{cmp}_{\mathcal{T}}(\mathrm{RO}_r^{(k)}[i],\ \mathrm{RO}_r^{(k)}[i+1]) = \mathrm{True}$.

Induction step. Consider inserting the $(k{+}1)$-st key x into the AVL tree rooted at ρ_k, using the binary-search-tree (BST) branching rule induced by $\mathrm{cmp}_{\mathcal{T}}$, followed by AVL rebalancing. Let ρ_{k+1} denote the root of the resulting AVL tree, and let $\mathrm{RO}^{(k+1)} := \mathrm{In}(\rho_{k+1})$ be its in-order traversal.

We first examine the moment immediately after inserting x and before AVL-tree rebalancing. At this moment, x is inserted as a *leaf*, and thus the sequence $\mathrm{RO}_r^{(k+1)}$ is obtained from $\mathrm{RO}_r^{(k)}$ by inserting the single new element x at exactly one position. Consequently, all adjacent pairs in $\mathrm{RO}_r^{(k+1)}$ remain unchanged from $\mathrm{RO}_r^{(k)}$ except those involving x. Let p and s denote the in-order predecessor and successor of x in $\mathrm{RO}_r^{(k+1)}$ (if they exist), that is, p immediately precedes x and s immediately follows x in the sequence $\mathrm{RO}_r^{(k+1)}$.

Predecessor. If p exists, then x, as a leaf node, lies in the right subtree of p in the tree before rebalancing. Hence the insertion search path for x must have visited the node storing p and taken the right branch at p. By the BST branching rule induced by cmp, this implies $\mathrm{cmp}_{\mathcal{T}}(p, x) = \mathrm{True}$.

Successor. If s exists, then x lies in the left subtree of s in the tree before rebalancing. Thus the insertion search path must have visited the node storing s and taken the left branch at s. By the branching rule, this implies $\mathrm{cmp}_{\mathcal{T}}(x, s) = \mathrm{True}$.

All other adjacent pairs in $\mathrm{RO}_r^{(k+1)}$ are identical to those in $\mathrm{RO}_r^{(k)}$ and therefore satisfy the property by the induction hypothesis. Finally, AVL-tree rebalancing consists of a constant number of local rotations, each of which preserves the in-order traversal sequence. Hence rebalancing does not change $\mathrm{RO}_r^{(k+1)}$, and the *Adjacent-Comparator Consistency* property remains true after inserting the $(k{+}1)$-st node. This completes the induction. □

Lemma 6.2 (Stitching preserves Adjacent-Comparator consistency in FEAT). *Fix a round r. Let RO_r be the round ordering produced by the in-order traversal of the AVL tree in round r, and let TailList be the tail list carried into round r. Let Merged_r be the output of the comparator-guided stitching procedure (two-pointer merge) that merges TailList and RO_r using $\mathrm{cmp}_{\mathcal{T}}$. Then Merged_r satisfies* Adjacent-Comparator Consistency*:*

$$\forall t, \quad \mathrm{cmp}_{\mathcal{T}}(\mathrm{Merged}_r[t],\ \mathrm{Merged}_r[t+1]) = \mathrm{True}.$$

Proof. We prove by considering every adjacent pair in Merged_r and showing that the consistency holds.

Invariant. At any time during the merge, if the last element appended to Merged_r is denoted by z, then for both current heads

TailList$[i]$ and $\mathrm{RO}_r[j]$, whenever they exist, it holds that

$$i \le |\textsc{TailList}| \Rightarrow \mathrm{cmp}_{\mathcal{T}}(z, \textsc{TailList}[i]) = \mathsf{True},$$

$$j \le |\mathrm{RO}_r| \Rightarrow \mathrm{cmp}_{\mathcal{T}}(z, \mathrm{RO}_r[j]) = \mathsf{True}.$$

We establish this invariant as follows: If the algorithm appends $\textsc{TailList}[i]$, it must be because $\mathrm{cmp}_{\mathcal{T}}(\textsc{TailList}[i], \mathrm{RO}_r[j]) = \mathsf{True}$. Moreover, since TailList already satisfies Adjacent-Comparator Consistency, we have the new last element $\textsc{TailList}[i]$ precedes the new tail head $\textsc{TailList}[i+1]$, i.e., $\mathrm{cmp}_{\mathcal{T}}(\textsc{TailList}[i], \textsc{TailList}[i+1]) = \mathsf{True}$. The case where the algorithm appends $\mathrm{RO}_r[j]$ is symmetric, using the fact that RO_r satisfies Adjacent-Comparator Consistency (Lemma 6.1). Therefore, the invariant holds throughout the merge.

Adjacent pairs in $\textsc{Merged}_r$. Consider any adjacent pair $(\textsc{Merged}_r[t], \textsc{Merged}_r[t{+}1])$. Element $\textsc{Merged}_r[t{+}1]$ is appended immediately after $\textsc{Merged}_r[t]$ in some merge iteration. At that iteration, $\textsc{Merged}_r[t{+}1]$ is one of the current heads of the two lists. By the invariant, immediately before appending $\textsc{Merged}_r[t{+}1]$, the last element in $\textsc{Merged}_r$ (which is exactly $\textsc{Merged}_r[t]$) satisfies $\mathrm{cmp}_{\mathcal{T}}(\textsc{Merged}_r[t], \textsc{Merged}_r[t{+}1]) = \mathsf{True}$. Hence every adjacent pair in $\textsc{Merged}_r$ is comparator-consistent. This proves that $\textsc{Merged}_r$ satisfies Adjacent-Comparator Consistency. □

Lemma 6.3 (Boundary consistency between FO_{r-1} and $\textsc{Merged}_r$). *Let* FO_{r-1} *be the final ordering after round* $r-1$*, and let* $\textsc{TailList}_{r-1}$ *be the tail list carried into round* r*. Let* z *be the last transaction of* FO_{r-1} *(if any), and let* m *be the head transaction of* $\textsc{Merged}_r$ *(if any). If both lists are non-empty, then* $\mathrm{cmp}_{\mathcal{T}}(z, m) = \mathit{True}$.

Proof. We consider two cases.

Case 1: m *is the head transaction of* $\textsc{TailList}_{r-1}$. Since $\textsc{TailList}_{r-1}$ is exactly the non-finalized suffix of the previous round's merged list, the head transaction of $\textsc{TailList}_{r-1}$ was the immediate successor of z at the end of round $r-1$. By Lemma 6.2, the previous merged list satisfies Adjacent-Comparator Consistency, and in particular the boundary pair between the last finalized transaction z and the head m of the merged list is consistent, i.e., $\mathrm{cmp}_{\mathcal{T}}(z, m) = \mathsf{True}$.

Case 2: m *is the head transaction of* RO_r. At the end of round $r-1$ the transaction m was still *blank* (i.e., not yet present as a non-blank transaction), otherwise it would have been included either in FO_{r-1} or in $\textsc{TailList}_{r-1}$. Since m is *blank* and z is *solid* by the end of round $r-1$, we have $\mathrm{score}(z) \ge \tau_{\mathrm{solid}}$ and $\mathrm{score}(m) < \tau_{\mathrm{shaded}}$, so the local orderings of more than $\tau_{\mathrm{solid}} - \tau_{\mathrm{shaded}} = \tau_{\mathrm{dec}}$ honest replicas have committed z but not m by the end of round $r-1$. Thus, $W_{r-1}(z, m) > \tau_{\mathrm{dec}} = \tau_{\mathrm{shaded}} > W_{r-1}(m, z)$, hence $\mathrm{cmp}_{\mathcal{T}}(z, m) = \mathsf{True}$.

Therefore, in both cases we have $\mathrm{cmp}_{\mathcal{T}}(z, m) = \mathsf{True}$, which completes the proof. □

Theorem 6.4 (Adjacent-Comparator Consistency of Final Ordering in FEAT). *The final transaction ordering* FO *produced by* FEAT *satisfies* Adjacent-Comparator Consistency*, i.e.,*

$$\forall t, \quad \mathrm{cmp}_{\mathcal{T}}(\mathrm{FO}[t], \mathrm{FO}[t+1]) = \mathit{True}.$$

Proof. We prove the claim by showing that throughout the execution of FEAT, whenever transactions are appended to the final ordering FO, the *Adjacent-Comparator Consistency* property is preserved for the updated final ordering.

Induction over rounds. Initially, FO is empty, and hence Adjacent-Comparator Consistency holds trivially.

Assume inductively that Adjacent-Comparator Consistency holds for FO_{r-1}. By Lemma 6.2, $\textsc{Merged}_r$ constructed in round r satisfies Adjacent-Comparator Consistency. By Lemma 6.3, the boundary pair between the last element of FO_{r-1} and the first element of $\textsc{Merged}_r$ is also comparator-consistent.

In round r, the updated final ordering FO_r is obtained by appending a prefix of $\textsc{Merged}_r$ to FO_{r-1}. Since the appended prefix is a contiguous segment of $\textsc{Merged}_r$, all adjacent pairs within it are consistent; and since the boundary pair with FO_{r-1} is consistent, the concatenation preserves Adjacent-Comparator Consistency.

Therefore, FO_r satisfies Adjacent-Comparator Consistency. By induction over r, the final transaction ordering FO produced by FEAT satisfies Adjacent-Comparator Consistency.

□

Lemma 6.5 (FEAT comparator is γ-induced). *Fix any two distinct transactions* $T_i \ne T_j$*. If at least a* γ*-fraction of honest replicas receive* T_i *before* T_j*, then the global comparator used by* FEAT *satisfies* $\mathrm{cmp}_{\mathcal{T}}(T_i, T_j) = \mathit{True}$.

Proof. We prove by contradiction. Assume that at least a γ-fraction of honest replicas receive T_i before T_j, but nevertheless $\mathrm{cmp}_{\mathcal{T}}(T_i, T_j) = \mathsf{False}$. By the definition of $\mathrm{cmp}_{\mathcal{T}}$ (via the first round in which the pair becomes decidable), the event $\mathrm{cmp}_{\mathcal{T}}(T_i, T_j) = \mathsf{False}$ implies that there exists some round r such that the opposite direction reaches the decidability threshold, i.e., $W_r(T_j, T_i) \ge \tau_{\mathrm{dec}}$. In FEAT, $\tau_{\mathrm{dec}} = \frac{n-f}{2}$.

Since at least a γ-fraction of honest replicas receive T_i before T_j, at most a $(1-\gamma)$-fraction of honest replicas can receive T_j before T_i. Moreover, Byzantine replicas (at most f) may contribute arbitrarily to the committed local orderings. Therefore, for every round r,

$$W_r(T_j, T_i) \le (1-\gamma)(n-f) + f.$$

Combining with $W_r(T_j, T_i) \ge \tau_{\mathrm{dec}} = \frac{n-f}{2}$ yields

$$(1-\gamma)(n-f) + f \;\ge\; \frac{n-f}{2}.$$

After rearrangement, we get

$$n \;\le\; \frac{(2\gamma+1)f}{2\gamma-1}.$$

This contradicts the system model requirement $n > \frac{(2\gamma+1)f}{2\gamma-1}$. Hence our assumption is false, and we conclude that $\mathrm{cmp}_{\mathcal{T}}(T_i, T_j) = \mathsf{True}$.

This proves that the global comparator $\mathrm{cmp}_{\mathcal{T}}$ is γ-induced. □

Theorem 6.6 (FEAT achieves γ-Batch-Order-Fairness). *The final transaction ordering* FO *produced by* FEAT *satisfies* γ*-Batch-Order-Fairness with respect to comparator* $\mathrm{cmp}_{\mathcal{T}}$.

Proof. By Lemma 6.5, the global comparator $\mathrm{cmp}_{\mathcal{T}}$ used by FEAT is γ-induced. By Theorem 6.4, the final transaction ordering FO produced by FEAT satisfies Adjacent-Comparator Consistency. Therefore, the conditions of Theorem 4.4 (Sufficiency of Adjacent-Comparator Consistency) hold. Applying Theorem 4.4, we conclude

that FO satisfies γ-Batch-Order-Fairness with comparator $\mathrm{cmp}_{\mathcal{T}}$. □

Theorem 6.7 (FEAT per-round upper bound). *Fix a round r. Assume that the number of non-blank transactions that participate in round r (i.e., the transactions inserted into the round-r AVL tree, and thus appearing in FO_r) is $O(|\mathcal{T}|)$. Then the number of comparator queries against $\mathrm{cmp}_{\mathcal{T}}$ made by* FEAT *in round r is $O(|\mathcal{T}|\log|\mathcal{T}|)$.*

Proof. In round r, FEAT inserts the $O(|\mathcal{T}|)$ non-blank transactions into an AVL tree. By standard AVL-tree properties, an AVL tree containing $O(|\mathcal{T}|)$ nodes has height $O(\log|\mathcal{T}|)$. Therefore, inserting a transaction follows a root-to-leaf search path of length $O(\log|\mathcal{T}|)$ and thus incurs $O(\log|\mathcal{T}|)$ comparator queries.

Although shaded transactions may be retried when the comparator returns $\perp$, the system model guarantees that for each transaction, within at most c rounds (for a constant c), the committed local orderings of all honest replicas will include this transaction. Consequently, any transaction first attempted in round $r' \le r$ can successfully inserted by round $r' + c$, with $O(c)$ attempts in total. Each insertion attempt follows a root-to-leaf path of length $O(\log|\mathcal{T}|)$, so the total number of comparator queries spent on the AVL-tree of round r is $O\big(|\mathcal{T}|\log|\mathcal{T}|\big)$.

This within-c-rounds visibility assumption is essential for the per-round bound. Without it, a shaded transaction whose pairwise precedence against some other tree node never becomes decidable would be deferred and retried indefinitely, so the per-round comparison count grows without bound. Even under the strictly weaker retry policy of re-inserting a shaded transaction only when a newly committed local ordering contains it, each shaded transaction may still be retried $O(n)$ times in the worst case, again exceeding the $O(|\mathcal{T}|\log|\mathcal{T}|)$ budget.

In the same round, FEAT stitches the round ordering FO_r with the persistent TailList using the two-pointer merge. Let $a := |\textsc{TailList}|$ and $b := |\mathrm{FO}_r|$. Each merge step performs exactly one comparator query between the two current head elements and advances at least one pointer. Hence the total number of comparator queries during stitching in round r is at most $a + b$. Under the assumption that the number of participating non-blank transactions in each round is $O(|\mathcal{T}|)$, we have both $a = O(|\mathcal{T}|)$ and $b = O(|\mathcal{T}|)$. Therefore, the stitching phase contributes $O(|\mathcal{T}|)$ comparator queries in round r, which is asymptotically dominated by $O(|\mathcal{T}|\log|\mathcal{T}|)$.

Combining the two parts, the total number of comparator queries performed by FEAT in round r is $O(|\mathcal{T}|\log|\mathcal{T}|)$. □

Corollary 6.8 (Tight per-round comparison bound). *Fix a round r and assume that the number of non-blank transactions that participate in round r is $\Theta(|\mathcal{T}|)$. Then* FEAT *has $\Theta(|\mathcal{T}|\log|\mathcal{T}|)$ expected comparison complexity in round r in the worst case over comparators.*

Proof. The upper bound $O(|\mathcal{T}|\log|\mathcal{T}|)$ follows from Theorem 6.7. For the matching lower bound, consider any comparison-based algorithm that outputs a final ordering satisfying Adjacent-Comparator Consistency. By Theorem 4.5, the lower bound is $\Omega(|\mathcal{T}|\log|\mathcal{T}|)$. Since FEAT is comparison-based and enforces Adjacent-Comparator Consistency within each round, this lower bound applies to FEAT as well. Combining the upper and lower bounds yields $\Theta(|\mathcal{T}|\log|\mathcal{T}|)$ expected comparison complexity in the worst case over comparators. □

Theorem 6.9 (Safety). *Assume (i) the consensus layer commits the same local-ordering fragments to every honest replica each round, and (ii) all honest replicas apply the same deterministic transaction-insertion order within each non-blank class (cf. the note at the end of Section 5). Then every honest replica running* FEAT *produces the same final ordering* FO.

Proof. By the system model (Section 3), in each round r the consensus layer commits the same set of local-ordering fragments to every honest replica. Every step performed by FEAT in round r is a deterministic function of the committed orderings up to round r and of FO and TailList carried from the previous round.

We prove by induction on r that at the end of round r, every honest replica holds an identical AVL tree, identical round ordering RO_r, identical TailList, and identical FO_r. The base case $r = 0$ is trivial, since TailList and FO_0 are initially empty at every replica. For the inductive step, both the committed input from the consensus layer to round $r + 1$ and the per-replica state inherited from round r are identical across honest replicas; applying the deterministic round procedure of FEAT to identical inputs yields identical outputs. Hence FO_{r+1}, $\textsc{TailList}_{r+1}$, and the AVL tree are identical at every honest replica. Taking the limit as $r \to \infty$, the final ordering FO is the same at every honest replica. □

7 Conclusion

In this paper, we resolved the per-round comparison complexity of enforcing γ-Batch-Order-Fairness. We introduced *Adjacent-Comparator Consistency* as a sufficient condition for γ-Batch-Order-Fairness with respect to a *γ-Induced Comparator*, and proved a lower bound of $\Omega(|\mathcal{T}|\log|\mathcal{T}|)$ comparator queries in the worst case over comparators via reduction from comparison-based sorting. We then presented FEAT, an AVL-tree-based fair-ordering algorithm that matches this bound by assembling the final ordering through balanced-BST insertions. The matching upper and lower bounds pin the per-round complexity at $\Theta(|\mathcal{T}|\log|\mathcal{T}|)$, and FEAT serves as a drop-in replacement for the ordering layer of existing BFT fair-ordering protocols.

Acknowledgments

This work is partially funded by NSF Award Number 2245373.

References

[1] 2022. Aptos Whitepaper: Safe, Scalable, and Upgradeable Web3 Infrastructure. https://arxiv.org/abs/2201.01107

[2] M AdelsonVelskii and Evgenii Mikhailovich Landis. 1962. *An algorithm for the organization of information.* Technical Report.

[3] Amit Agarwal, Kushal Babel, Sourav Das, and Babak Poorebrahim Gilkalaye. 2025. Time-Lock Encrypted Storage for Blockchains. *Cryptology ePrint Archive* (2025).

[4] Orestis Alpos, Ignacio Amores-Sesar, Christian Cachin, and Michelle Yeo. 2024. Eating Sandwiches: Modular and Lightweight Elimination of Transaction Reordering Attacks. In *27th International Conference on Principles of Distributed Systems (OPODIS 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 286)*, Alysson Bessani, Xavier Défago, Junya Nakamura, Koichi Wada, and

Yukiko Yamauchi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 12:1–12:22. doi:10.4230/LIPIcs.OPODIS.2023.12

[5] Mohammad Javad Amiri, Chenyuan Wu, Divyakant Agrawal, Amr El Abbadi, Boon Thau Loo, and Mohammad Sadoghi. 2024. The Bedrock of Byzantine Fault Tolerance: A Unified Platform for BFT Protocols Analysis, Implementation, and Experimentation. In *21st USENIX Symposium on Networked Systems Design and Implementation, NSDI 2024, Santa Clara, CA, April 15-17, 2024*, Laurent Vanbever and Irene Zhang (Eds.). USENIX Association, 371–400.

[6] Avi Asayag, Gad Cohen, Ido Grayevsky, Maya Leshkowitz, Ori Rottenstreich, Ronen Tamari, and David Yakira. 2018. A fair consensus protocol for transaction ordering. In *2018 IEEE 26th International Conference on Network Protocols (ICNP)*. IEEE, 55–65.

[7] Kushal Babel, Andrey Chursin, George Danezis, Lefteris Kokoris-Kogias, and Alberto Sonnino. 2023. Mysticeti: Low-Latency DAG Consensus with Fast Commit Path. *CoRR* abs/2310.14821 (2023).

[8] Brad Bachu, Joel Hasbrouck, Fahad Saleh, and Xin Wan. 2025. An overview of uniswap v4 for researchers. *Available at SSRN 5152215* (2025).

[9] Sam Blackshear, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Xun Li, Mark Logan, Ashok Menon, Todd Nowacki, Alberto Sonnino, et al. 2024. Sui lutris: A blockchain combining broadcast and consensus. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2606–2620.

[10] Felix Brandt, Vincent Conitzer, Ulle Endriss, Jérôme Lang, and Ariel D. Procaccia. 2016. *Handbook of Computational Social Choice*. Cambridge University Press, Cambridge, UK.

[11] Lorenz Breidenbach, Christian Cachin, Benedict Chan, Alex Coventry, Steve Ellis, Ari Juels, Farinaz Koushanfar, Andrew Miller, Brendan Magauran, Daniel Moroz, et al. 2021. Chainlink 2.0: Next steps in the evolution of decentralized oracle networks. *Chainlink Labs* 1 (2021), 1–136.

[12] Vitalik Buterin. 2013. Ethereum White Paper: A Next-Generation Smart Contract and Decentralized Application Platform. https://ethereum.org/en/whitepaper/.

[13] Christian Cachin, Klaus Kursawe, Frank Petzold, and Victor Shoup. 2001. Secure and efficient asynchronous broadcast protocols. In *Annual International Cryptology Conference*. Springer, 524–541.

[14] Christian Cachin, Jovana Mićić, Nathalie Steinhauer, and Luca Zanolini. 2022. Quick order fairness. In *International Conference on Financial Cryptography and Data Security*. Springer, 316–333.

[15] Miguel Castro and Barbara Liskov. 2002. Practical Byzantine Fault Tolerance and Proactive Recovery. *ACM Trans. Comput. Syst.* 20, 4 (2002), 398–461. doi:10.1145/571637.571640

[16] Marquis de Condorcet. 1785. *Essay on the Application of Analysis to the Probability of Majority Decisions*. Imprimerie Royale, Paris.

[17] Andrei Constantinescu, Diana Ghinea, Lioba Heimbach, Zilin Wang, and Roger Wattenhofer. 2024. A Fair and Resilient Decentralized Clock Network for Transaction Ordering. In *27th International Conference on Principles of Distributed Systems (OPODIS 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 286)*, Alysson Bessani, Xavier Défago, Junya Nakamura, Koichi Wada, and Yukiko Yamauchi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 8:1–8:20. doi:10.4230/LIPIcs.OPODIS.2023.8

[18] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. 2020. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *2020 IEEE symposium on security and privacy (SP)*. IEEE, 910–927.

[19] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2022. Narwhal and Tusk: a DAG-based mempool and efficient BFT consensus. In *Proceedings of the Seventeenth European Conference on Computer Systems*. ACM, 34–50. doi:10.1145/3492321.3519594

[20] Pranav Garimidi, Joachim Neu, and Max Resnick. 2025. Multiple Concurrent Proposers: Why and How. *arXiv preprint arXiv:2509.23984* (2025).

[21] Neil Giridharan, Florian Suri-Payer, Ittai Abraham, Lorenzo Alvisi, and Natacha Crooks. 2024. Autobahn: Seamless high speed BFT. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 1–23.

[22] Suyash Gupta, Jelle Hellings, and Mohammad Sadoghi. 2021. RCC: Resilient Concurrent Consensus for High-Throughput Secure Transaction Processing. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*. IEEE, 1392–1403. doi:10.1109/ICDE51399.2021.00124

[23] Suyash Gupta, Dakai Kang, Dahlia Malkhi, and Mohammad Sadoghi. 2025. Carry the Tail in Consensus Protocols. *arXiv preprint arXiv:2508.12173* (2025).

[24] Dakai Kang, Junchao Chen, Tien Tuan Anh Dinh, and Mohammad Sadoghi. 2025. FairDAG: Consensus Fairness over Multi-Proposer Causal Design. *Proc. VLDB Endow.* 19, 2 (Oct. 2025), 265–278. doi:10.14778/3773749.3773763

[25] Dakai Kang, Suyash Gupta, Dahlia Malkhi, and Mohammad Sadoghi. 2025. HotStuff-1: Linear Consensus with One-Phase Speculation. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–29. doi:10.1145/3725308

[26] Dakai Kang, Sajjad Rahnama, Jelle Hellings, and Mohammad Sadoghi. 2024. SpotLess: Concurrent Rotational Consensus Made Practical through Rapid View Synchronization. In *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, Netherlands, May 13-17, 2024*. IEEE.

[27] Alireza Kavousi, Duc V. Le, Philipp Jovanovic, and George Danezis. 2026. BlindPerm: Efficient MEV Mitigation with an Encrypted Mempool and Permutation. In *29th International Conference on Principles of Distributed Systems (OPODIS 2025) (LIPIcs)*. Schloss Dagstuhl. doi:10.4230/LIPIcs.OPODIS.2025.36

[28] Mahimna Kelkar, Soubhik Deb, Sishan Long, Ari Juels, and Sreeram Kannan. 2023. Themis: Fast, strong order-fairness in byzantine consensus. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 475–489.

[29] Mahimna Kelkar, Fan Zhang, Steven Goldfeder, and Ari Juels. 2020. Order-fairness for byzantine consensus. In *Advances in Cryptology–CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part III 40*. Springer, 451–480.

[30] Klaus Kursawe. 2020. Wendy, the good little fairness widget: Achieving order fairness for blockchains. In *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies*. 25–36.

[31] Rujia Li, Xuanwei Hu, Qin Wang, Sisi Duan, and Qi Wang. 2025. Transaction fairness in blockchains, revisited. *IEEE Transactions on Dependable and Secure Computing* (2025).

[32] Dahlia Malkhi and Pawel Szalachowski. 2023. Maximal Extractable Value (MEV) Protection on a DAG. In *4th International Conference on Blockchain Economics, Security and Protocols (Tokenomics 2022) (Open Access Series in Informatics (OASIcs), Vol. 110)*, Yackolley Amoussou-Guenou, Aggelos Kiayias, and Marianne Verdier (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 6:1–6:17. doi:10.4230/OASIcs.Tokenomics.2022.6

[33] Benjamin Marsh, Steven Landers, and Jayendra Jog. 2025. Sei Giga. *arXiv preprint arXiv:2505.14914* (2025).

[34] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. 2016. The honey badger of BFT protocols. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 31–42.

[35] Heena Nagda, Shubhendra Pal Singhal, Mohammad Javad Amiri, and Boon Thau Loo. 2024. Rashnu: Data-Dependent Order-Fairness. *Proceedings of the VLDB Endowment* 17, 9 (2024), 2335–2348.

[36] Satoshi Nakamoto. 2009. Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf

[37] Ben Riva, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2025. Seahorse: Efficiently Mixing Encrypted and Normal Transactions. In *Financial Cryptography and Data Security (FC 2025)*. Springer. doi:10.1007/978-3-032-07035-7_3

[38] Mohammad Sadoghi. 2025. *The Problems of Consensus: An Ethical Inquiry into Democratic and Decentralized Principles*. Springer. doi:10.1007/978-3-032-07282-5

[39] Fabian Schär. 2021. Decentralized finance: On blockchain-and smart contract-based financial markets. *FRB of St. Louis Review* (2021).

[40] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2022. Bullshark: DAG BFT Protocols Made Practical. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM, 2705–2718.

[41] Chrysoula Stathakopoulou, Signe Rüsch, Marcus Brandenburger, and Marko Vukolić. 2021. Adding fairness to order: Preventing front-running attacks in bft protocols using tees. In *2021 40th International Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 34–45.

[42] Shaokang Xie, Dakai Kang, Hanzheng Lyu, Jianyu Niu, and Mohammad Sadoghi. 2025. Fides: Scalable Censorship-Resistant DAG Consensus via Trusted Components. *arXiv preprint arXiv:2501.01062* (2025).

[43] Anatoly Yakovenko. 2018. Solana: A new architecture for a high performance blockchain v0. 8.13. *Whitepaper* (2018).

[44] Sen Yang, Fan Zhang, Ken Huang, Xi Chen, Youwei Yang, and Feng Zhu. 2024. SoK: MEV Countermeasures. In *Proceedings of the Workshop on Decentralized Finance and Security* (Salt Lake City, UT, USA) *(DeFi '24)*. Association for Computing Machinery, New York, NY, USA, 21–30. doi:10.1145/3689931.3694911

[45] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT Consensus with Linearity and Responsiveness. In *Proceedings of the ACM Symposium on Principles of Distributed Computing*. ACM, 347–356. doi:10.1145/3293611.3331591

[46] Pouriya Zarbafian and Vincent Gramoli. 2023. Lyra: Fast and scalable resilience to reordering attacks in blockchains. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 929–939.

[47] Dirk A Zetzsche, Douglas W Arner, and Ross P Buckley. 2020. Decentralized finance. *Journal of Financial Regulation* 6, 2 (2020), 172–203.

[48] Yunhao Zhang, Srinath Setty, Qi Chen, Lidong Zhou, and Lorenzo Alvisi. 2020. Byzantine ordered consensus without byzantine oligarchy. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 633–649.